

Q1 Ideation

15 points | Manual work, agent opportunities, input/output contracts

The problem at NeuronForge

Product managers turn scattered meetings, briefs and stakeholder notes into PRDs and backlog items. The repeated work is not just writing: it is finding evidence, preserving conflicting views, filling the template and checking that every story still reflects approved scope. PRD Genie organizes those steps while keeping product decisions with a human reviewer.

1. Collect and reconcile source material

Manual step: locate relevant notes, identify versions and distinguish current decisions from old proposals. Agent solution: ingest named documents, retain source identity and retrieve relevant context. Inputs: transcripts, briefs, notes, template and prior PRDs. Outputs: source manifest, retrieval evidence and a pinned current-product catalog. Risks: stale or unrelated documents and instructions embedded in source text. Controls: explicit product identity, current-versus-style-reference labels and treating document content as data.

2. Extract requirements from messy notes

Manual step: reread narrative text, capture exact thresholds, identify personas and separate requirements from assumptions. Agent solution: Requirement Extractor produces typed requirements, facts and issues with exact source quotes. Inputs: source documents or the reviewed sample catalog. Outputs: stable evidence IDs, functional and non-functional requirements, uncertainty and provenance. Risks: omitted criteria, changed numbers and invented scope. Controls: source-quote validation, schema checks, coverage checks and a reviewer-visible packet.

3. Surface gaps and stakeholder tensions

Manual step: detect missing ownership, dates, metrics and unresolved design tradeoffs. Agent solution: Gap Analyzer produces questions linked to existing evidence. Inputs: extracted requirements, facts and issues. Outputs: unresolved questions and stakeholder-resolution requests. Risks: treating compatible preferences as contradictions or silently choosing a design. Controls: no product decision in the gap stage, valid evidence-ID constraints and explicit reviewer resolution.

Q1 Ideation continued

15 points | Document generation, stories and responsible automation

4. Review and complete requirements

Manual step: circulate notes and establish what was actually approved. Agent solution: persist a review ID, revision and source hash; ask for missing details and invalidate prior approval when answers alter evidence. Inputs: extracted packet and actual reviewer answers. Outputs: complete approved revision and audit receipt. Risks: stale approval and a typed name mistaken for authenticated identity. Controls: exact-revision checks now; authenticated reviewer identity is a production requirement.

5. Produce a complete PRD

Manual step: rewrite the same facts into a standard ten-section document. Agent solution: PRD Generator maps approved evidence into the provided template, including detailed Mermaid architecture. Inputs: approved catalog, metadata, template and style context. Outputs: wiki.md with source references, assumptions, resolved questions and timeline. Risks: a polished document hiding unsupported claims or missing fields. Controls: deterministic rendering, required-field checks and final user document review.

6. Create implementation-ready backlog items

Manual step: split scope into epics and stories, write descriptions and acceptance criteria, then recheck coverage. Agent solution: Story Breakdown groups approved requirements and preserves GIVEN, WHEN and THEN scenarios. Inputs: approved requirements and generated PRD mappings. Outputs: three epics, eight RelayOps stories, Markdown, JSON and Jira-import CSV. Risks: invented benefits, merged personas or altered criteria. Controls: requirement-ID coverage and exact approved scenario transfer. No Jira issues are created automatically.

7. Version and learn from results

Manual step: save versions, collect feedback and diagnose poor outputs. Agent solution: atomic GitHub publication when connected, plus native Langflow traces and case-level evaluation. Inputs: generated artifacts, review receipt and run metadata. Outputs: version commit, trace evidence and reproducible result files. Risks: credential leakage, duplicate commits and claiming quality from execution alone. Controls: secret references, idempotent manifests, non-force branch updates and separate execution, automated-quality and human-review states.

Prioritization

The core path is extraction, PRD generation and story breakdown. Gap Analysis is the selected extended capability; RAG and human review strengthen it. Fine-tuning is an optional experiment with prepared data, not a completed training claim.

Q2 Program charter

15 points | Vision, objectives, scope and accountability

Vision and mission

Make product documentation consistent, traceable and easier to review without transferring scope decisions to an LLM. The capstone demonstrates this with a fictional RelayOps Webhook Delivery Console and real author/reviewer identity supplied by Raghavendra Kaushik.

Objectives and deliverables

Deliver sequential specialist agents; exact source grounding; the complete supplied PRD template; gap analysis; reviewer approval before strict generation; detailed Mermaid diagrams; import-ready Jira stories; observability; all twelve baseline outcomes; and a reproducible submission package. Store knowledge and submission files in Google Drive and version artifacts on the authorized GitHub feature branch.

In scope

Local Langflow with the existing Groq connection; a free-text evaluation workflow; a separate strict catalog workflow with BM25 retrieval context and complete pinned source catalog; durable revision-bound review; Markdown and CSV export; implementation code and workflow JSON; eight detailed synthetic knowledge files; evaluation and training preparation.

Scope boundaries

The strict catalog path does not yet automatically normalize arbitrary narratives. The live demonstration uses an explicit embedded knowledge snapshot. Direct Drive retrieval and server-side GitHub auto-commit await credentials. Jira imports are prepared, not executed. A supported fine-tuning connection and label review are still needed. Enterprise SSO, role-backed reviewer authentication, retention policy and multi-user operational hardening are outside this demonstrated release.

Owner and stakeholders

Raghavendra Kaushik is capstone owner, document author, actual requirements reviewer and final merge decision-maker. The course assessor reviews submission evidence and determines points. Future PM/TPM pilot users, engineering, security and platform owners are proposed organizational roles, not named or committed project staff. Fictional RelayOps characters belong only to the sample product knowledge base.

Q2 Success and measurement

15 points | Acceptance criteria and business validation

Artifact acceptance

The strict sample must contain all ten template sections, eight functional requirements, four NFRs and sixteen GIVEN/WHEN/THEN scenarios. Every story must map to an approved requirement; three epics and eight stories must appear in the Jira import. Author and reviewer must be populated from user input. Missing required facts must block generation and prompt for an answer. The final sample contains no empty, TBD or Not Stated fields.

Grounding and review acceptance

Exact source references, approved-revision integrity and template completeness are machine checked. A human must assess omissions, unsupported implications and usability. Every amendment must create a new revision and invalidate stale approval. Requirements approval permits draft generation; final PRD acceptance and merge remain separate. No machine test establishes a universal no-hallucination guarantee.

Evaluation acceptance and observed status

Record every official baseline input and its actual output or failure. T11 and T12 must evaluate the real T1 artifacts. This requirement is met by twelve result records: four automated passes and eight failures, including two execution errors. Production quality promotion is not met. The strict sample passed its deterministic checks, and the v2 review/export suite passed 22 software tests; these are distinct evidence sets.

Proposed productivity study

Measure time from input collection to human-approved draft for ten representative cases, once manually and once with assistance, counterbalancing order to reduce learning effects. Proposed success: at least 50% lower median elapsed drafting time without worse factual support or omission rate. Record review time separately, report distributions and outliers, and include failed runs. No such business-impact measurement has yet been performed.

Operational targets for a pilot

Before a multi-user pilot, require authenticated reviewer identity, persistent review storage, a tested scoped publishing connection, secret scanning, recovery testing and trace retention rules. Establish latency and cost budgets after representative runs; do not convert the single fast sample into an SLA. Block promotion until the supplied failing cases are repaired and a fresh held-out set is reviewed.

Q2 Timeline, risks and rollout

15 points | Demonstrated milestones and proposed next gates

Demonstrated sequence

September 5-12: source review, prototype workflow construction, Groq connection and earlier T1/T9 execution evidence. September 13: complete RelayOps catalog, live review and approved generation, versioned feature-branch artifacts, full baseline capture, assignment writeups and submission packaging. These are implementation milestones, not proof of a production launch.

Proposed next iteration

Week 1 after connection access: connect server-side Drive and GitHub, confirm credential scoping and test one version commit without force. Week 2: address baseline omissions, classification and execution failures; evaluate a new domain-separated holdout. Week 3: conduct the paired PM timing and source-review study. Week 4: consider a broader rollout only after quality, identity, recovery and retention gates pass. These relative weeks are a proposal, not promised dates.

Risk register and mitigation

Hallucination or omission: preserve quotes, reject invalid references and require semantic review. Source gaps: ask focused questions and block strict generation. Provider quota/rate limits: use the configured working provider, bounded retries and explicit failure states. Stale knowledge: track file identity and catalog revisions. Review persistence loss: mount the database on a durable Docker volume and test restore. Credential exposure: secret fields, scoped access and clean exports. Duplicate publication: manifest comparison and atomic non-force commit. Evaluation leakage: separate diagnostic baseline from new holdouts.

Rollout and change control

Start with one capstone owner and synthetic knowledge. Next invite a small PM pilot after unresolved quality and connection gates pass. Record requested changes against evidence IDs, regenerate only after approval and keep final merge human controlled. Expand users and document types only after failure recovery and source-grounding metrics remain acceptable. Roll back by restoring the prior workflow version and retaining existing approved document versions.

Decision log

Use the existing local Langflow Docker environment because the n8n trial expired. Use the available Groq inference connection after OpenAI quota errors. Keep the repository private on Feature/Raghav_IK_Capstone_SEP2026. Prepare Jira imports. Use Raghavendra as actual reviewer for this capstone. Store the final submission in Capstone_Raghav_SEP2026 on Google Drive.

Q3 Build PRD Genie - page 1 of 2

45 points | Design 10; core capabilities 12; extended capability 8

Architecture and orchestration

A sequential handoff makes each agent decision inspectable. The free-text workflow runs Extractor -> Gap Analyzer -> PRD Generator -> Story Builder with schema, exact-quote and reference validation. Empty or ambiguous evidence returns questions and skips document generation. The strict v2 workflow runs retrieval -> extraction -> completeness -> persisted human review -> PRD mapping -> stories -> optional GitHub publication -> output. See the separate architecture diagram and exported JSON for full branching.

Agent decisions and tool rationale

The Extractor distinguishes stated requirements, facts and uncertainty. Gap Analysis asks questions without deciding product scope. The PRD stage maps evidence to the supplied template. Story Breakdown groups existing requirements and preserves acceptance criteria. Langflow provides an inspectable low-code canvas and native traces; Groq GPT-OSS 120B uses the existing working connection. Python custom components provide deterministic validation and rendering. n8n is an optional alternative export, not the demonstrated runtime.

Human-in-the-loop implementation

Review state stores a review ID, revision and content hash. Missing reviewer identity initially blocked strict generation. Raghavendra supplied the name and approved source revision 2, hash 6a35b08e...86af9e. Generation then produced wiki.md and Jira stories. Answers that change content require new approval. This is an evidence-revision control; a typed name is not enterprise authentication. Final PRD review and merge into main remain human actions.

Core and extended evidence

The approved fictional RelayOps sample contains ten template sections, eight FRs, four NFRs, sixteen GWT scenarios, three epics and eight stories, with an eleven-node/sixteen-edge Mermaid product diagram. Gap Analysis is the selected extended capability. RAG adds BM25 context from eight detailed files while pinning the complete current catalog. Past PRD content supplies style, not current facts. Direct Drive retrieval is not live-connected; the demonstrated run uses the explicit snapshot. Arbitrary text-to-v2-catalog normalization and actual fine-tuning remain uncompleted.

Reproducibility and exports

Import the reviewed Langflow JSON, bind GROQ_API_KEY and follow the displayed review actions. A separate tested baseline export includes the repaired evidence-ID schema. The source kit contains tests, prompts, schemas, review state logic, publishing code and training preparation. README specifies dependencies, connection gaps and reproduction steps; no secrets are embedded.

Q3 Build PRD Genie - page 2 of 2

45 points | Observability 5; baseline results 5; cost/evaluation 5

Connected observability

Native Langflow traces record root and component spans in its database. The approved-generation trace is 49393cd3-6cbc-46c7-a9a0-f6ea08f5a96e, with a 2.98-second root span in the trace view. The PRD and story component spans show about 1.34 and 1.21 seconds. Component-reported timing uses a different measurement boundary; it is not substituted for root latency. Canvas, in-action and trace screenshots are included. External Langfuse is not configured.

Official baseline outcomes

All twelve supplied inputs have recorded artifacts or explicit error records. Automated passes: T1, T9, T11 and T12. Failures: T2 misses users clarification; T3 misses the conflict label; T4 loses criteria classification/transfer; T5 uses ambiguous rather than insufficient; T6 and T8 fail execution; T7 fails exact NFR classification; T10 fails the dependency/owner check. T11 and T12 evaluate the actual T1 artifacts, not separate calls. T1 is September 12 evidence; other cases were captured September 13. Human semantic review of this suite is pending.

Diagnostic iteration and evidence limits

The first T2 run failed because Gap Analysis referenced an unknown evidence ID. Constraining the schema to known IDs allowed the retry to complete, but its semantic check still failed. The failed attempt is retained. Markdown for T1 was deterministically reconstructed from captured validated structure; T5 required restoring quote escaping from rendered UI text. These capture limitations are documented. Two generic UI failures lack a captured component error and are not represented as successful outputs.

Observed cost and future budget

Groq list prices checked September 13 are \$0.15 per million input tokens and \$0.60 per million output tokens. Approved PRD generation used 374 input/321 output tokens; stories used 597/292. The combined estimate is $(971 \times 0.15 + 613 \times 0.60) / 1,000,000 = \0.00051345 for generation only. Extraction, retries, storage and hosting are additional. This is not a billing total or end-to-end cost. Track per-stage token usage and include failures in future run-level budgets.

Evaluation strategy and sources

Separate execution success, contract validity, semantic support, completeness and business impact. Keep all baseline failures; add source-level gold labels and fresh domain-separated holdouts before promotion. Compare base and fine-tuned extraction only after label review and an actual training job. Primary references: docs.langflow.org/traces; console.groq.com/docs/models; console.groq.com/docs/model/openai/gpt-oss-120b. Rubric: supplied Problem Statement, Assignment and Submission Guidelines.

Q4 Reflection

5 points | One-page reflection grounded in captured evidence

What the traces taught me

The most useful failure was an unknown evidence ID emitted by Gap Analysis. The validator stopped the pipeline instead of accepting an unsupported link. A dynamic enum narrowed the model to available evidence IDs and the retry ran successfully. Yet the reporting case still omitted a required clarification about users. This separates a repaired execution contract from a repaired product-quality outcome.

What the baseline revealed

The recorded suite has four automated passes and eight failures. Some failures preserve much of the source meaning but miss the required classification: incomplete notes were labelled ambiguous, and the technical case did not satisfy the exact NFR requirement. The export case is more consequential because explicit acceptance criteria failed to reach stories as criteria. Two runs returned execution errors. A polished output or green component status alone would not reveal these weaknesses.

How I would improve the system

First, recover detailed component traces for the failed runs and introduce narrowly scoped recovery for invalid references and malformed outputs. Next, strengthen acceptance-criterion extraction, persona separation, dependency ownership and missing-question coverage, then rerun the supplied cases while retaining earlier failures. Evaluate a fresh held-out domain set to avoid confusing baseline-specific prompt tuning with generalization. Finally, connect Drive and GitHub server credentials, harden reviewer authentication and test durable review-state recovery.

Risks of AI-generated PRDs

An invented threshold can become an engineering commitment; an omitted exception can become a security defect; an inferred date can be mistaken for a promise. The complete RelayOps catalog and deterministic renderer reduce unsupported additions, but no single architecture proves universal absence of hallucinations. Human review must check factual support and omissions as well as readability. Fictional sample metrics must remain clearly labelled and must never be presented as measured business outcomes.

Connection to evaluation skills

This work makes evaluation part of product delivery: define expected behavior before judging the model, distinguish source fidelity from fluency, inspect failures at the stage where they arise, and keep mechanical checks separate from semantic assessment. Fine-tuning needs reviewed labels, an actual training run and a held-out comparison; prepared examples alone prove none of those. The next release decision should depend on improved evidence, not a more confident narrative. This reflection was drafted with AI assistance for Raghavendra to review before final submission.